EEE 4601 · COMPLEX ENGINEERING PROBLEM

TuneTrace

Play it. We'll name it.



A real-time audio pattern recognition system for music — built end-to-end in MATLAB.

Introduction :-

Group Members :-

1. Ahbab Muhtasim – 220021111
2. Md Abid Sarwar Khan – 220021122
3. Mahib Abtahi – 220021130
4. Riasat Zaman – 220021144
5. Juhayer Mahdiul Alam Ashfi – 220021145

A real-time audio pattern recognition system for music — built end-to-end in MATLAB.

THE PROBLEM

You know the tune. You just can't name it.

01

No title, no artist

Only the melody survives in memory — lyrics and metadata are gone.

02

Real-world noise

Clips arrive pitch-shifted, tempo-warped, and picked up over background hiss.

03

Needs to feel instant

A useful assistant has to respond in near real time, not minutes.

EXPECTED OUTCOME

Three zones, one seamless pipeline

ZONE 1

Build the database

YouTube playlist → pre-processing → feature extraction → fingerprint index

ZONE 2

Capture the query

Mic or file → pre-processing → fingerprinting, on the fly

ZONE 3

Search & display

Matching engine → ranked song → similar tracks → GUI

UNDER THE HOOD

Every clip walks the same five steps

1
Pre-process
Mono, 16kHz, trim & normalise

2
Spectrum
FFT frames, noise-floor removal

3
Fingerprint
Chroma + landmark hashes

4
Match
Hash voting + DTW alignment

5
Rank
Top-K songs + similar tracks

Auto mode trusts strong landmark evidence first, and quietly falls back to melody matching when there isn't any — so the user never has to choose.

THE ALGORITHMS AT WORK

Four stages of code turn a clip into a name

Feature extraction

Hann-windowed FFT spectrogram feeds a mel filterbank and 12-bin chroma — hand-coded, no toolbox.

Landmark + DTW matching

Sorted hash-merge join votes on time offset; open-end subsequence DTW aligns chroma by cosine distance.

Confidence scoring

Hash and melody scores blend into one combined score; auto mode trusts whichever evidence is stronger.

Recommendations

Summary features are z-score standardized, L2-normalized, then ranked by cosine similarity.

Every transform — FFT, mel filters, DTW, hashing — is implemented in base MATLAB, no Audio Toolbox.

Two ears are better than one

♫ LANDMARK HASHING

"That exact recording."



- Spectral peaks become a fingerprint
- Fingerprint matched straight to the database
- Shazam-style — fast & noise-tolerant

♪ CHROMA + DTW

"Same song, different playback."



- Pitch content folded into a melody shape
- Aligned against the reference, any starting point
- Forgives pitch shifts & tempo drift

Same two ears, under the hood

♫ LANDMARK HASHING

The exact-match path

- Top-K log-magnitude peaks/frame, min bin spacing enforced
- 32-bit hash: 8-bit f1 + 8-bit f2 + 16-bit frame gap
- Sorted hash-merge join finds shared hashes, votes on time offset
- Score = 0.75 × vote ratio + 0.25 × hash coverage

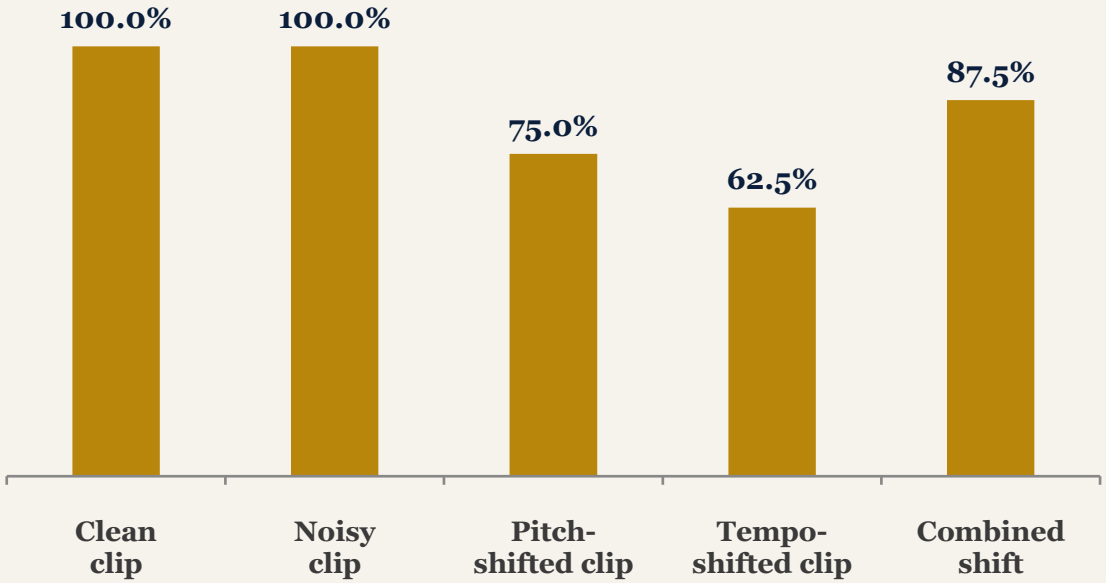
♪ CHROMA + DTW

The melody-match path

- 12-bin chroma: median-subtracted, log-compressed, L2-normalized
- Adjacent frames averaged (beat-scale smoothing), length capped
- All 12 circular pitch shifts tested to find the best key alignment
- Open begin/end subsequence DTW, cosine frame distance

PUT TO THE TEST

85% top-1 accuracy across 5 hostile conditions



85.0%

overall top-1
(34 / 40 queries)

~0.12s

mean matching time
per query — real time

HONEST NOTES

Where it shines, and what's next

✓ Strengths

- Near-perfect on original / noisy clips
- Tolerates key changes via chroma search
- ~0.12s per query — feels instant
- Zero external toolbox dependency

→ Next up

- Tempo-shifted clips are the weak spot (62.5%)
- Fix: beat-synchronous chroma aggregation
- Scale fix: inverted hash index for lookup
- Grow catalogue beyond course scale



From a few rough seconds of audio

to a named song, in moments.

Thank you — questions welcome.